Gateway DCIM: monitoramento de data center

Sistemas distribuídos com multicast, UDP, gRPC e TCP, todas as mensagens em Protocol Buffers

Antonio Marcos Justino Carneiro · matrícula **511911**

TI0151 Sistemas Distribuídos · Universidade Federal do Ceará · 2026.2 · Prof. Paulo A. L. Rego

Domínio: data center (DCIM)

Um sistema de gerenciamento da infraestrutura acompanha temperatura, energia e refrigeração dos racks.

Gateway

Ponto central do DCIM: descobre as fontes, guarda as leituras, comanda o ar-condicionado e responde ao operador.

Fontes

Ar-condicionado (controlável, gRPC)
Sensor de temperatura por rack (evento ao passar do limiar)
PDU (consumo em watts por rack)

Cliente

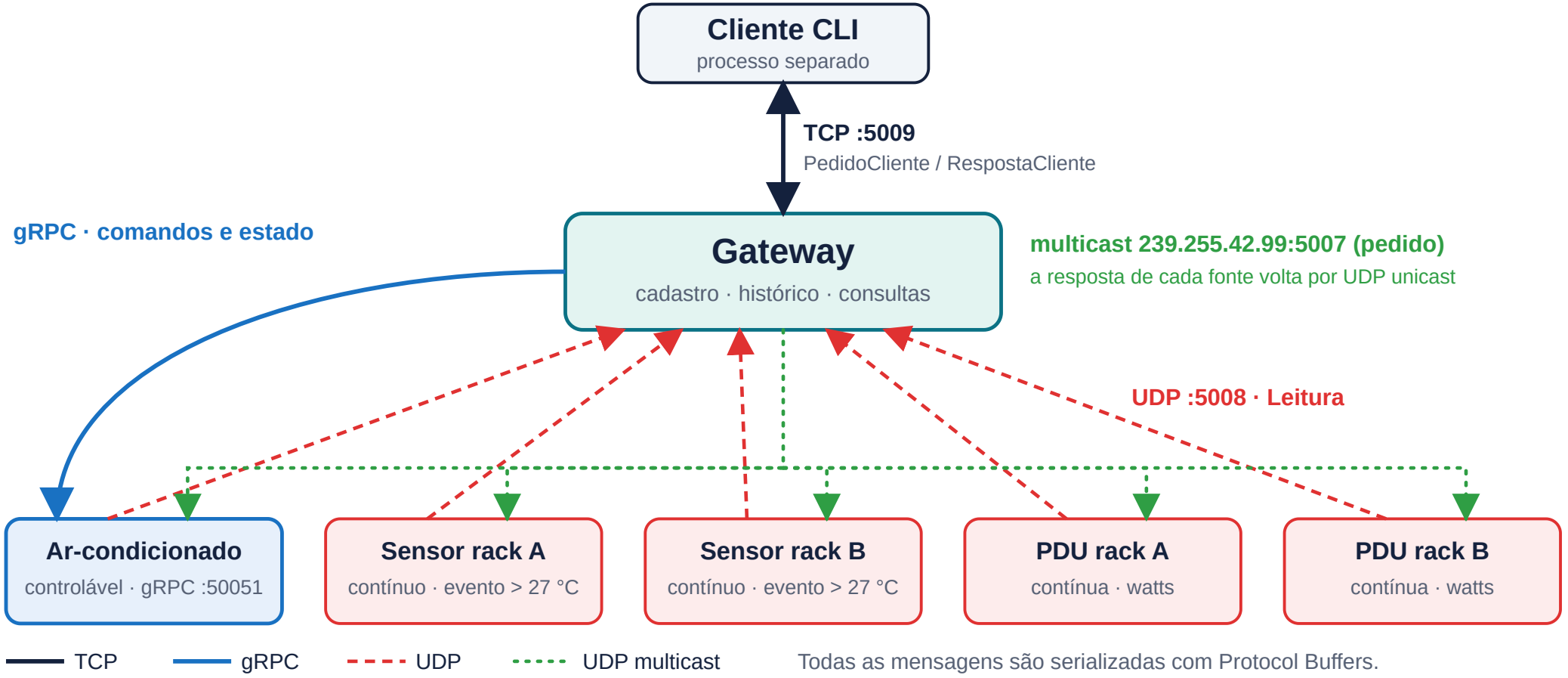
Linha de comando: lista fontes e estados, liga/desliga, ajusta setpoint e faz 3 consultas agregadas.

Limiar de alerta: 27 °C, topo da faixa recomendada (18–27 °C) do ASHRAE TC 9.9, *Thermal Guidelines*, 5ª ed. (2021). A faixa permitida da classe A1 vai até 32 °C.

Unidades: °C e W · cada fonte é um processo separado · 5 fontes na demonstração (1 ar-condicionado, 2 sensores, 2 PDUs).

Arquitetura

O gateway é servidor para o cliente e para as leituras, e cliente gRPC da fonte controlável.



Linguagem, frameworks e bibliotecas

Uma única linguagem, sem frameworks de aplicação: sockets da biblioteca padrão, Protobuf e gRPC.

Camada	Tecnologia
Linguagem	Java 21 (Temurin)
Build	Maven 3.8 · `protobuf-maven-plugin` 0.6.1 baixa o `protoc` e o plugin gRPC e gera o código a cada `mvn compile`
Serialização	Protocol Buffers (proto3) · `protobuf-java` 3.25.5 · um único arquivo `datacenter.proto` compartilhado por todos os processos
Invocação remota	gRPC Java 1.68.0 · `grpc-netty-shaded`, `grpc-protobuf`, `grpc-stub`
Sockets	`java.net`: `MulticastSocket` e `DatagramSocket` (UDP), `ServerSocket` e `Socket` (TCP)
Concorrência	Threads de `java.lang` e `ScheduledExecutorService`; estado compartilhado com `synchronized` e `volatile`

Processos (classes com `main`): `Gateway`, `ArCondicionado`, `SensorTemperatura`, `Pdu`, `Cliente` · apoio: `Cadastro`, `DescobertaFonte`, `Quadro`, `Rede`, `Config`

Descoberta UDP multicast

O gateway não conhece os IPs das fontes: pergunta ao grupo `239.255.42.99:5007`; cada fonte responde por unicast.

```
// gateway → grupo multicast
message DescobertaSolicitacao {
  string gateway_ip         = 1;
  uint32 gateway_porta_udp  = 2;
  uint32 nonce              = 3;
}

// fonte → gateway (unicast)
message DescobertaResposta {
  string fonte_id    = 1;
  TipoFonte tipo     = 2;
  string ip          = 3;
  uint32 porta       = 4;  // gRPC (0 = sem)
  bool controlavel   = 5;
  string rack        = 6;
  uint32 intervalo_ms = 7;
  optional EstadoAr estado_inicial = 8;
  uint32 nonce       = 9;
}
```

- Ao iniciar, o gateway envia o pedido; **repete a cada 30 s** para achar fontes que subiram depois ou perderam o pedido (UDP).
- A **solicitação leva o IP e a porta UDP do gateway**: a fonte aprende para onde mandar as leituras, sem configuração.
- A resposta vai ao endereço de origem do pedido. O **nonce** descarta respostas de rodadas anteriores.
- A resposta informa tipo, IP, porta (a do gRPC, nas controláveis) e o estado inicial.
- Socket de descoberta separado do de leituras: o datagrama Protobuf não traz o tipo da mensagem.

Leituras UDP

Um datagrama = uma mensagem `Leitura`, enviada à porta 5008 do gateway.

```
message Leitura {
  string fonte_id      = 1;
  TipoFonte tipo       = 2;
  string rack          = 3;
  Grandeza grandeza    = 4;  // °C ou W
  double valor         = 5;
  int64 timestamp_ms   = 6;
  bool evento          = 7;  // envio imediato
  uint32 intervalo_ms  = 8;
  uint64 seq           = 9;
  optional EstadoAr estado_ar = 10;
}
```

- **Periódico:** a cada 2 s (intervalo anunciado).
- **Evento imediato:** o sensor envia na hora ao **cruzar** 27 °C, subindo ou descendo (`evento = true`), sem repetir enquanto permanece acima.
- `intervalo_ms` alimenta o critério de inatividade; `seq` permite detectar perda, duplicação e reordenação.
- O ar-condicionado anexa o `estado_ar`; **desligado, continua enviando** com `ligado = false`: ausência de leituras significa falha.
- UDP porque as leituras são frequentes e a perda de uma é tolerável.
- O gateway usa a **hora de chegada**, não o `timestamp_ms` da fonte (relógios diferentes).

Serviço gRPC do ar-condicionado gRPC

A fonte controlável é o servidor; o gateway é o cliente. Definição em `datacenter.proto`.

```
service ArCondicionado {
  rpc Configurar(ConfigurarRequest) returns (EstadoAr);
  rpc ObterEstado(Vazio)            returns (EstadoAr);
}

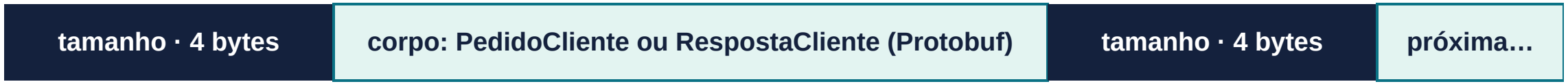
message ConfigurarRequest {
  optional bool   ligado         = 1;
  optional double setpoint_c     = 2;  // 10..35
  optional uint32 ventilador_pct = 3;  // 0..100
  optional uint32 intervalo_ms   = 4;  // 500..60000
}

message EstadoAr {
  bool ligado = 1;  double setpoint_c = 2;
  uint32 ventilador_pct = 3;  uint32 intervalo_ms = 4;
}
```

- **Atualização parcial:** `optional` distingue "ausente" de zero; ausente = não alterar.
- **Idempotente:** o comando carrega valores absolutos, então repetir após um timeout é seguro (diferente de "diminuir 2 graus").
- **Validação na fonte:** valor fora da faixa devolve `INVALID_ARGUMENT`; o gateway repassa a mensagem ao cliente.
- **Prazo de 3 s** por chamada (`withDeadlineAfter`); fonte fora do ar devolve `UNAVAILABLE`.
- Stub bloqueante no gateway; o servidor usa o pool de threads do gRPC.

Cliente ↔ gateway TCP

TCP é um fluxo de bytes sem fronteiras de mensagem: cada mensagem Protobuf é enquadrada com seu tamanho.



Inteiro de 4 bytes em big-endian com o tamanho do corpo, seguido do corpo.

```
message PedidoCliente {
  oneof pedido {
    ListarFontesRequest listar   = 1;
    ComandoRequest      comando  = 2;  // fonte_id + config
    ConsultaRequest     consulta = 3;  // tipo + janela_s
  }
}
message RespostaCliente {
  bool   ok   = 1;
  string erro = 2;
  oneof resposta {
    ListarFontesResponse listar   = 3;
    ComandoResponse      comando  = 4;
    ConsultaResponse     consulta = 5;
  }
}
```

- Leitura com `readInt()` e `readFully()`: um `read()` solto poderia devolver só parte do quadro.
- **Conteúdo inválido:** o fluxo segue alinhado; resposta `ok = false` e a conexão continua.
- **Tamanho absurdo** (limite 1 MiB): alinhamento perdido e risco de esgotar a memória; conexão fechada.
- Uma thread por cliente conectado.

Três consultas agregadas

Calculadas pelo gateway sobre as leituras que chegaram nos últimos *N* segundos (janela padrão: 300 s).

Consulta	Definição	Resultado na demonstração (janela 8 s)
`consulta media`	Média das leituras de temperatura dos sensores de rack (°C). O ar-condicionado não entra: mede o ar de saída.	25,35 °C (8 amostras)
`consulta rack`	Rack com a maior **média** de temperatura na janela e o valor dessa média.	28,72 °C, rack A (sensor A esquentado)
`consulta pico`	Maior leitura de potência das PDUs (W) e a PDU que a gerou.	6457,79 W, pdu-rack-B (pico simulado)

- **Sem leituras na janela:** a resposta traz `tem_dados = false` e o cliente exibe "sem dados"; testado antes de as PDUs serem descobertas.
- A **média por rack** mostra a tendência de aquecimento e evita alarme por uma amostra ruidosa; o **pico instantâneo** já é coberto pelo evento do sensor.
- O histórico fica em memória, limitado a 1 hora.

Concorrência em cada processo

Cada atividade fica bloqueada esperando (datagrama, comando, relógio); threads permitem esperar em paralelo.

Processo	Threads
Gateway	`udp-leituras` (servidor UDP) · `tcp-clientes` (aceita conexões, **1 thread por cliente**) · agenda de descoberta (multicast a cada 30 s) · cliente gRPC chamado dentro da thread do cliente TCP
Ar-condicionado	principal (simula a temperatura e envia leituras) · `descoberta` (multicast) · **pool do gRPC** (atende `Configurar`)
Sensor / PDU	principal (amostragem e envio) · `descoberta` · `teclado` (apenas para forçar situações na demonstração)
Cliente	uma thread: envia o pedido e espera a resposta

- **Estado compartilhado protegido:** `Cadastro` (fontes e histórico) com todos os métodos `synchronized`; estado do ar-condicionado também; `volatile` nos campos lidos entre threads (destino das leituras, base da temperatura simulada).
- O comando via gRPC altera o estado na thread do gRPC, e a thread principal o lê no envio seguinte: sem isso haveria leitura de valor pela metade.

Falhas e robustez

Comportamentos testados com os processos reais, inclusive encerrando o ar-condicionado durante a execução.

Situação	Comportamento
Fonte sem nenhuma leitura por mais de **3 intervalos**	`INATIVA` na listagem; volta a `ATIVA` na próxima leitura
Comando a fonte que caiu	gRPC `UNAVAILABLE`; o cliente recebe "fonte não respondeu"; o gateway segue
Comando fora da faixa (setpoint 99)	`INVALID_ARGUMENT` da fonte, repassado ao cliente
Datagrama UDP inválido ou sem campos essenciais	Descartado (Protobuf aceita quase qualquer bytes: o conteúdo também é validado)
TCP: conteúdo inválido / pedido vazio	Resposta `ok = false`; a conexão continua
TCP: tamanho declarado de 2 GiB	Conexão fechada; o gateway continua atendendo os demais
Fonte que sobe depois do gateway	Achada na rodada seguinte de descoberta (até 30 s)

Limite conhecido: o detector por tempo não distingue fonte caída de rede caída (sistema assíncrono); reduzir a tolerância detecta mais rápido, com mais falsos alarmes.

Decisões, limitações e próximos passos

Escopo mínimo do enunciado atendido; os extras ficaram de fora.

Requisitos atendidos

✓ Multicast ao iniciar, respostas com tipo, IP, porta e estado

✓ 3 tipos de fonte, processos separados; 1 controlável

✓ UDP periódico + envio imediato por limiar

✓ gRPC definido em `.proto` com estado de volta

✓ Cliente TCP/Protobuf com 3 consultas

✓ Falha de fonte demonstrada

Limitações

- Gateway é ponto único de falha; reiniciá-lo perde o histórico.
- Sem TLS no gRPC nem autenticação no TCP.
- Detecção de falha por tempo (falsos positivos em rede instável).
- Multicast restrito à rede local (TTL 1).

Próximos passos (extras)

- Persistência do histórico (arquivo ou banco).
- Cliente com interface gráfica e séries temporais.
- Fontes em outra linguagem, reaproveitando o mesmo `.proto`.